

Supply Service and How it Works ?

- In our case, cabs are the supply services and they will be tracked by geolocation (latitude and longitude).
- All the active cabs keep on sending the location to the server once every 4 seconds through a web application firewall and load balancer.
- The accurate GPS location is sent to the data center through Kafka's Rest APIs once it passes through the load balancer. Here we use **Apache Kafka** as the data hub.
- Once the latest location is updated by Kafka it slowly passes through the respective worker nodes' main memory.
- Also, a copy of the location (state machine/latest location of cabs) will be sent to the database and to the dispatch optimization to keep the latest location updated.
- We also need to track a few more things such as the number of seats, the presence of a car seat for children, the type of vehicle, can a wheelchair be fit, and allocation (for example, a cab may have four seats but two of those are occupied.)

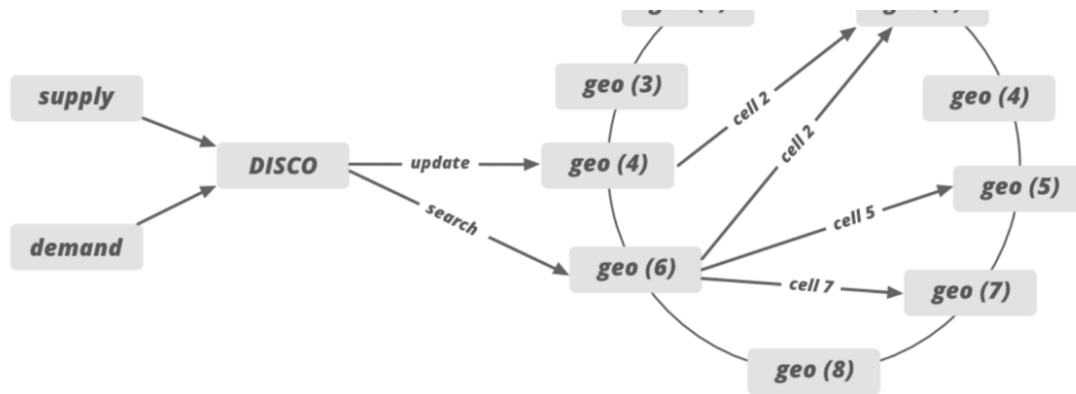
Demand Service and How it Works ?

- Demand service receives the request of the cab through a web socket and it tracks the GPS location of the user. It also receives different kinds of requirements such as the number of seats, type of car, or pool car.
- Demand gives the location (cell ID) and user requirement to supply and make requests for the cabs.

How Dispatch System Match the Riders to drivers ?

- We have discussed that DISCO divides the map into tiny cells with a unique ID. This ID is used as a sharding key in DISCO. When supply receives the request from demand the location gets updated using the cell ID as a shard key.
- These tiny cells' responsibilities will be divided into different servers lies in multiple regions (consistent hashing).
- For example, we can allocate the responsibility of 12 tiny cells to 6 different servers (2 cells for each server) lying in 6 different regions.





- Supply sends the request to the specific server based on the GPS location data. After that, the system draws the circle and filters out all the nearby cabs which meet the rider's requirements.
- After that, the list of the cab is sent to the ETA to calculate the distance between the rider and the cab, not geographically but by the road system.
- The sorted ETA is then sent back to the supply system to offer to a driver.

If we need to handle the traffic for the newly added city then we can increase the number of servers and allocate the responsibilities of newly added cities' cell IDs to these servers.

How to Scale the Dispatch System?

The dispatch system (including supply, demand, and web socket) is built on **NodeJS**. NodeJS is the asynchronous and event-based framework that allows you to send and receive messages through WebSockets whenever you want.

*Uber uses an open-source **ringpop** to make the application cooperative and scalable for heavy traffic. Ring pop has mainly three parts and it performs the below operation to scale the dispatch system.*

- It maintains **consistent hashing** to assign the work across the workers. It helps in sharding the application in a way that's scalable and fault-tolerant.
- Ringpop uses **RPC (Remote Procedure Call) protocol** to make calls from one server to another server.
- Ringpop also uses a **SWIM membership protocol/gossip protocol** that allows independent workers to discover each other's responsibilities. This way each server/node knows the responsibility and the work of other nodes.
- Ringpop detects the newly added nodes to the cluster and the node which is removed from the cluster. It distributes the loads evenly when a node is added or removed.

How does the Uber Defines a Map Region?

Before launching a new operation in a new area, Uber onboarded the new region to the map technology stack. In this map region, we define various subregions labeled with grades A, B, AB, and C.

- **Grade A:** This subregion is responsible to cover the urban centers and commute areas. Around 90% of Uber traffic gets covered in this subregion, so it's important to build the highest quality map for subregion A.
- **Grade B:** This subregion covers the rural and suburban areas which are less populated and less traveled by Uber customers.
- **Grade AB:** A union of grade A and B subregions.
- **Grade C:** Covers the set of highway corridors connecting various Uber Territories.

How Does UBER Build the Map?

Uber uses a third-party map service provider to build the map in their application. Earlier Uber was using Mapbox services but later Uber switched to Google Maps API to track the location and calculate ETAs.

1. Trace coverage

- Trace coverage spot the missing road segments or incorrect road geometry.
- Trace coverage calculation is based on two inputs: map data under testing and historic GPS traces of all Uber rides taken over a certain period of time.
- It covers those GPS traces onto the map, comparing and matching them with road segments.
- If we find missing road segments (no road is shown) on GPS traces then we take some steps to fix the deficiency.

2. Preferred access (pick-up) point accuracy

We get the pickup point in our application when we book the cab in Uber. Pick-up points are a really important metric in Uber, especially for large venues such as airports, college campuses, stadiums, factories, or companies. We calculate the distance between the actual location and all the pickup and drop-off points used by drivers.

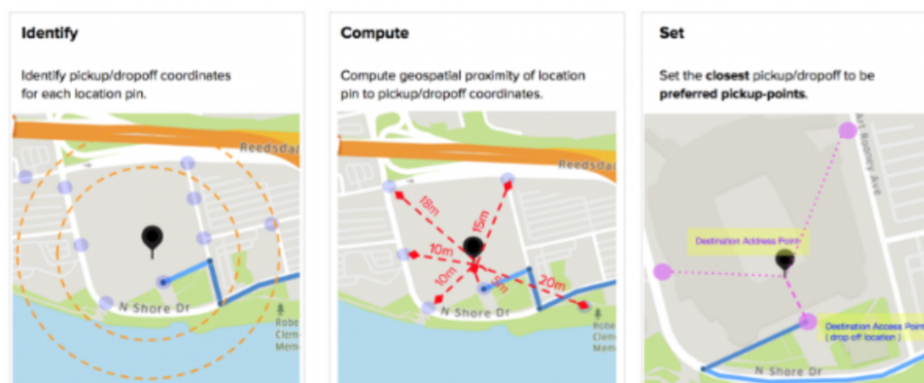


Figure 4: Refining access points is composed of three main steps: identify actual pick-up and drop-off locations used by drivers for a place or address (left), compute the distances from those locations to the place or address (middle), and then set the preferred access point based on the shortest distance (right).

Image Source: <https://www.uber.com/blog/maps-metrics-computation/>

- The shortest distance (closest pickup point) is then calculated and we set the pin to that location as a preferred access point on the map.
- When a rider requests the location indicated by the map pin, the map guides the driver to the preferred access point.
- The calculation continues with the latest actual pick-up and drop-off locations to ensure the freshness and accuracy of the suggested preferred access points.
- Uber uses machine learning and different algorithms to figure out the preferred access point.

How ETAs Are Calculated?

ETA is an extremely important metric in Uber because it directly impacts ride-matching and earnings.

- ETA is calculated based on the road system (not geographically) and there are a lot of factors involved in computing the ETA (like heavy traffic or road construction).
- When a rider requests a cab from a location the app not only identifies the free/idle cabs but also includes the cabs which are about to finish a ride.

It may be possible that one of the cabs which are about to finish the ride is closer to the demand than the cab which is far away from the user. So many Uber cars on the road send GPS locations every 4 seconds, so to predict traffic we can use the driver's app's GPS location data.

- We can represent the entire road network on a graph to calculate the ETAs. We can use AI-simulated algorithms or simple [Dijkstra's algorithm](#) to find out the best route in this graph.
- In that graph, nodes represent intersections (available cabs), and edges represent road segments.
- We represent the road segment distance or the traveling time through the edge weight. We also represent and model some additional factors in our graph such as one-way streets, turn costs, turn restrictions, and speed limits.

Once the data structure is decided we can find the best route using Dijkstra's search algorithm which is one of the best modern routing algorithms today. For faster performance, we also need to use OSRM (Open Source Routing Machine) which is based on contraction hierarchies.